

Controle de Gastos Pessoais

Relatório do projeto — esqueleto e implementação das Aulas 1 a 6 (transações, categorias, saldo, resumo, filtros avançados com testes automatizados, exportação CSV, dashboard e preparação para deploy)

Python 3.11+

FastAPI

SQLite

Pydantic

De onde partimos

JÁ EXISTIA

- `specs/` com as specs das 6 aulas do projeto
- `docs/prompts-projeto1-controle-gastos.md`, o roteiro de prompts da aula
- `.claude/commands/implementar-spec.md`, o slash command `/implementar-spec`
- Nenhum código da aplicação ainda

OBJETIVO DA SESSÃO

- Criar o esqueleto do projeto + `CLAUDE.md`
- Implementar as Aulas 1 a 5 (`specs/aula-1-transacoes.md` a `aula-5-filtros-testes.md`)
- Verificar critérios de aceite e commitar a cada aula

Fluxo seguido em cada etapa: Plano → Aprovação → Implementação → Verificação → Commit

Esqueleto do projeto + CLAUDE.md

ESTRUTURA CRIADA

```
SDD1/
├─ CLAUDE.md
├─ requirements.txt
├─ .gitignore
├─ app/
│   ├── main.py → app + startup + handler 422
│   ├── database.py → conexão + init_db()
│   ├── models.py → schemas Pydantic
│   └─ routers/
├─ docs/ (já existia)
├─ specs/ (já existia)
└─ .claude/ (já existia)
```

CLAUDE.MD DOCUMENTA

- Descrição do projeto e stack
- Como rodar: venv + `pip install` + `uvicorn --reload`
- Estrutura de pastas e papel de cada módulo
- Convenções (datas, valores, tipo, erro 422)
- Seção "Decisões registradas" para acumular ao longo das aulas

Convenções e arquitetura

CONVENÇÕES DO PROJETO

- Datas em ISO `YYYY-MM-DD`
- Valores monetários com 2 casas decimais
- `tipo` sempre `"entrada"` ou `"saida"`
- Erros de validação em HTTP 422, corpo `{ "erro": "..."}`
- Banco SQLite criado na inicialização, se não existir

DECISÕES TÉCNICAS

- Acesso via `sqlite3` puro (sem ORM) — controle direto do SQL para filtros dinâmicos e agregações das próximas aulas
- Erro 422 tratado globalmente com um `exception_handler` de `RequestValidationError` — não repetido rota a rota
- `init_db()` chamado no evento `startup` do FastAPI

Registrar e listar transações

ENDPOINTS IMPLEMENTADOS

- `POST /transacoes` → 201 com a transação criada (com `id`)
- `GET /transacoes` → lista ordenada por `data` decrescente

MODELO DE DADOS

- `id` (auto), `data`, `descricao`, `valor`, `tipo`

ARQUIVOS ALTERADOS

- `app/database.py` — tabela `transacoes`, `criar_transacao`, `listar_transacoes`
- `app/models.py` — `TransacaoCreate`, `Transacao`
- `app/routers/transacoes.py` — rotas POST/GET (novo)
- `app/main.py` — registra o router

PASSO 3

Verificação contra os critérios de aceite

CRITÉRIO	RESULTADO
POST /transacoes retorna 201 com o objeto e id	✓ Passou
GET /transacoes lista mais recentes primeiro	✓ Passou
Transações sobrevivem ao reinício do servidor	✓ Passou
Erros de validação em 422 no formato {"erro": "..."}	✓ Passou

Testado ponta a ponta: servidor subido, transações criadas com datas diferentes, servidor reiniciado e histórico conferido, além de 3 cenários inválidos (valor ≤ 0 , tipo inválido, data malformada).

Versionando o slash command

PROBLEMA ENCONTRADO

O repositório tem um `.gitignore` na raiz que ignora `.claude/` em **todo** o repositório — política já existente, usada para não versionar configs locais.

Isso também excluía `SDD1/.claude/commands/implementar-spec.md`, que não é config local: é parte do fluxo de trabalho do projeto.

SOLUÇÃO

Exceção adicionada ao `.gitignore` raiz, escopada só a este projeto:

→ Reabre `SDD1/.claude/`

→ Reignora tudo dentro, exceto `commands/`

Resto do repositório continua ignorando `.claude/` normalmente.

CRUD completo com validação

ENDPOINTS IMPLEMENTADOS

- `PUT /transacoes/{id}` → 200 com o objeto atualizado, ou 404
- `DELETE /transacoes/{id}` → 204, ou 404
- PUT reaproveita o schema do POST — mesmas regras de validação valem para os dois

VALIDAÇÃO DE ESCRITA

- `valor > 0`, `tipo` e `data` já vinham da Aula 1
- `descricao` não vazia é nova: `field_validator` que dá `strip()` e rejeita string vazia

ARQUIVOS ALTERADOS

- `app/database.py` — `atualizar_transacao`, `remover_transacao`
- `app/models.py` — validador de `descricao` em `TransacaoCreate`
- `app/routers/transacoes.py` — rotas PUT / DELETE (novas)

Verificação — Aula 2

CRITÉRIO	RESULTADO
<code>PUT /transacoes/{id}</code> altera os campos e reflete em <code>GET /transacoes</code>	✓ Passou
<code>DELETE /transacoes/{id}</code> remove; buscar depois retorna 404	✓ Passou
<code>valor≤0</code> , tipo inválido, data malformada ou <code>descricao</code> vazia → 422, em POST e PUT	✓ Passou
Operar sobre id inexistente retorna 404 (PUT e DELETE)	✓ Passou

Testado ponta a ponta: criação, edição, listagem, tentativa de operar em id inexistente, 6 cenários inválidos (em POST e PUT) e exclusão seguida de nova tentativa (404).

Aula 2 — o que ficou definido

VALIDAÇÃO REAPROVEITADA

Como PUT usa o mesmo schema `TransacaoCreate` do POST, todas as regras (incluindo a nova validação de `descricao`) valem para os dois sem duplicar código.

FORMATO DO 404

PUT / DELETE em id inexistente usam o formato padrão do FastAPI (`{"detail": "..."}`).

O `{"erro": ...}` do CLAUDE.md é uma convenção só para 422 de validação — não foi estendida a outros status sem a spec pedir.

Categorias e filtro por categoria

ENDPOINTS IMPLEMENTADOS

- `POST /categorias` → 201; nome duplicado → 422
- `GET /categorias` → lista
- `GET /transacoes?categoria=<nome ou id>` → filtra
- `POST / PUT /transacoes` passam a aceitar `categoria_id`

ARQUIVOS ALTERADOS

- `app/database.py` — tabela `categorias`, migração idempotente de `transacoes`, nova exceção `ErroDominio`
- `app/models.py` — `CategoriaCreate` / `Categoria`, `categoria_id` em `TransacaoCreate`
- `app/routers/categorias.py` (novo) e ajustes em `transacoes.py`
- `app/main.py` — handler de `ErroDominio` → 422

Verificação — Aula 3

CRITÉRIO	RESULTADO
Criar categorias e associá-las a transações funciona	✓ Passou
<code>GET /transacoes?categoria=alimentacao</code> (nome ou id) retorna só as dessa categoria	✓ Passou
Categoria inexistente no filtro (nome ou id) → lista vazia, sem quebrar	✓ Passou
Categoria com nome repetido → 422 <code>{"erro": "..."} </code>	✓ Passou
Compatibilidade: transação sem <code>categoria_id</code> continua funcionando	✓ Passou

Bônus testado: `categoria_id` inexistente ao criar transação → 422 (violação de FK), não erro 500. Migração do banco já existente aplicada sem perda de dados.

Aula 3 — as duas ambiguidades da spec

(A) CATEGORIA_ID É OPCIONAL

A própria spec exige manter compatibilidade com as transações da Aula 1/2, que não têm categoria. Torná-lo obrigatório exigiria migrar dados antigos para algo que nenhum critério de aceite pede.

(B) CATEGORIA INEXISTENTE → LISTA VAZIA, NÃO 404

`GET /transacoes?categoria=...` é uma busca numa coleção, não a busca de um recurso por id. Lista vazia também evita ambiguidade quando a Aula 5 combinar esse filtro com período e valor.

Saldo e resumo mensal

ENDPOINTS IMPLEMENTADOS

- `GET /saldo` → {entradas, saidas, saldo} de tudo
- `GET /resumo?mes=YYYY-MM` → totais do mês +
`por_categoria`
- `mes` validado por regex (`Query(pattern=...)`) —
formato inválido cai no handler de 422 já existente

CÁLCULO

- Tudo via `SUM` / `GROUP BY` no SQL — nenhum laço Python
somando linha a linha

ARQUIVOS ALTERADOS

- `app/database.py` — `calcular_saldo`,
`calcular_totais_mes`, `resumo_por_categoria`
- `app/models.py` — `Saldo`, `Resumo`, `CategoriaResumo`
- `app/routers/resumo.py` (novo)

Verificação — Aula 4

CRITÉRIO	RESULTADO
<code>GET /saldo</code> retorna entradas, saídas e saldo corretos	✓ Passou
<code>GET /resumo?mes=2026-08</code> retorna totais e quebra por categoria corretos	✓ Passou
Mês sem transações → zeros e <code>por_categoria: []</code> , sem erro	✓ Passou
<code>mes</code> malformato → 422 <code>{"erro": "..."} </code>	✓ Passou

Testado com os dados de seed (5 categorias, 10 transações em jun/jul/ago de 2026) — valores conferidos manualmente para os 3 meses com dados, mais um mês vazio e dois formatos de `mes` inválidos.

Aula 4 — o que é "gasto" no resumo?

A spec pede a quebra de "gastos" por categoria — leio isso como só as transações `saida` (gasto = despesa); entradas não entram nessa quebra, mas continuam nos totais do mês normalmente.

Transações sem `categoria_id` também ficam fora de `por_categoria` (não há o que agrupar), mas continuam nos totais gerais. `categoria` no resultado é o **nome**, não o id — mais legível num resumo.

Filtros avançados e testes automatizados

FILTROS IMPLEMENTADOS

- `GET /transacoes` ganha `data_inicio`, `data_fim`, `valor_min`, `valor_max` — todos opcionais, combináveis com `categoria` por E lógico
- Query dinâmica com parâmetros vinculados (`?`) — nunca concatena valor de usuário no SQL
- Período/valor invertidos → 422 `{"erro": "..."}`

TESTES AUTOMATIZADOS (NOVO)

- `pytest` + `TestClient` — `tests/test_api.py`
- Fixture `client` (`tests/conftest.py`) troca `database.DB_PATH` por banco temporário — isolado do `gastos.db`
- `conftest.py` de raiz garante que `app` seja importável

Verificação — Aula 5

CRITÉRIO	RESULTADO
categoria + período + faixa de valor combinados → exatamente as que satisfazem tudo	✓ Passou
Cada filtro isolado funciona sozinho	✓ Passou
Período invertido → 422	✓ Passou
Valores inválidos (faixa invertida, não numérico) → 422	✓ Passou
Suíte pytest cobre os fluxos principais e passa integralmente	✓ Passou — 29/29

Revisão por subagente contra a spec

O QUE O SUBAGENTE ENCONTROU

- Nenhum bug de correção (query paramétrica correta, bordas `==` tratadas certo)
- Lacunas de cobertura: filtros só testados em combinação, nunca isolados; filtro de valor não excluía nada no teste combinado; faltava caso de parâmetro malformato

O QUE MUDOU DEPOIS

- +6 testes: `data_inicio` / `data_fim` / `valor_min` / `valor_max` isolados, parâmetro malformato
- `test_filtros_combinados` ganhou uma 4ª transação para o filtro de valor de fato excluir alguém
- Suíte foi de 23 → 29 testes

Aula 5 — período/valor invertidos como 422

`data_inicio > data_fim` e `valor_min > valor_max` não são erros de tipo (o Pydantic já valida isso) — são regras de negócio sobre os parâmetros da busca.

O router de `GET /transacoes` retorna `JSONResponse(422, {"erro": ...})` manualmente para os dois casos, mesmo formato do handler global. Não criamos uma exceção nova só para isso — `ErroDominio` continua reservado a violações detectadas pelo SQLite (nome duplicado, FK inválida).

Projeto migrado para repositório próprio

O QUE MUDOU

- Extraído do monorepo do curso via `git subtree split` — histórico completo preservado (11 commits, sem squash, sem perder a evolução aula a aula)
- Novo repositório público:
`github.com/fernandojsmelo/controle-de-gastos-pessoais-api`
- Projeto agora vive em
`~/Projetos Claude/controle-de-gastos-pessoais-api`, fora do monorepo

VERIFICAÇÃO

- `SDD1/` removido do monorepo original (código + a exceção órfã que existia só pra liberar `.claude/commands/` do `.gitignore` raiz)
- Testado de forma standalone: `venv novo`, `pip install -r requirements.txt`, 29/29 testes passando, fora do monorepo
- Hashes de commit mudaram (subtree split reescreve os commits) — a tabela de histórico já usa os hashes deste repositório

Exportação CSV, dashboard e deploy

ENDPOINTS IMPLEMENTADOS

- `GET /export.csv` → extrato em CSV, aceitando os mesmos filtros da Aula 5
- `GET /` e `GET /dashboard` → dashboard HTML (gráfico por categoria + saldo atual)

DEPLOY

- `python -m app.main` lê `HOST` / `PORT` do ambiente
- `DATABASE_PATH` configura o arquivo SQLite fora da máquina local

ARQUIVOS ALTERADOS

- `app/routers/export.py` (novo) — CSV via `csv` da `stdlib`
- `app/routers/dashboard.py` (novo) + `app/static/dashboard.html` — Chart.js via CDN
- `app/routers/transacoes.py` — filtros extraídos para `resolver_transacoes()`
- `app/database.py` — `DB_PATH` via `DATABASE_PATH`
- `app/main.py` — registra routers + entrypoint `__main__`
- `.env.example` (novo) — documenta as variáveis de ambiente

Verificação — Aula 6

CRITÉRIO	RESULTADO
<code>GET /export.csv</code> baixa um CSV válido com as colunas das transações; filtros aplicados afetam o conteúdo	✓ Passou
O dashboard exibe o gráfico de gastos por categoria e o saldo atual, consumindo os endpoints existentes	✓ Passou
O app roda fora da máquina local (deploy), com a porta e configs vindo de variáveis de ambiente	✓ Passou

Testado ponta a ponta: `pytest` com 33/33 (4 testes novos: CSV com/sem filtro, filtro inválido, dashboard em `/` e `/dashboard`); servidor subido manualmente com `PORT=5050 DATABASE_PATH=/tmp/deploy_test.db python -m app.main`, endpoints conferidos via `curl` e banco criado no caminho indicado.

Aula 6 — o que ficou definido

FILTROS COMPARTILHADOS

A validação de filtros da Aula 5 (incluindo os 422 de período/valor invertidos) foi extraída para `resolver_transacoes()`, usada por `GET /transacoes` e `GET /export.csv` — evita duas implementações divergentes do mesmo filtro.

COLUNAS DO CSV

`id, data, descricao, valor, tipo, categoria_id` — os mesmos campos do schema `Transacao`, sem juntar o nome da categoria.

DASHBOARD SEM ENDPOINT NOVO

O HTML consome `GET /saldo` e `GET /resumo?mes=<mês atual>` (calculado no cliente) — a spec pede consumir "os endpoints existentes", então nenhuma agregação nova foi criada no backend.

DEPLOY VIA VARIÁVEL DE AMBIENTE

`HOST / PORT` e `DATABASE_PATH` viram configuráveis, com os mesmos defaults de desenvolvimento. Não há credenciais no projeto; `.env.example` documenta sem valores sensíveis.

Commits deste repositório

COMMIT	MENSAGEM
22b7df4	Renomeia relatorio-sessao.pdf para relatorio-projeto.pdf
4d78f05	Adiciona Aula 6: exportação CSV, dashboard e preparação para deploy
cc56fc1	Atualiza relatório em PDF com a migração para repositório próprio
bfba840	Atualiza relatório em PDF com a Aula 5
c6a2ab8	Adiciona Aula 5: filtros avançados e testes automatizados
2d990c4	Atualiza relatório em PDF com a Aula 4
dff4d1c	Adiciona Aula 4: saldo e resumo mensal
d9405a2	Atualiza relatório em PDF com a Aula 3
485ab4c	Adiciona Aula 3: categorias e filtro por categoria
c85d248	Atualiza relatório em PDF com a Aula 2 e corrige quebra de linha das árvores
c013cd3	Adiciona Aula 2: CRUD completo com validação
4bb18c1	Adiciona relatório em PDF da sessão (esqueleto + Aula 1)
40b1862	Versiona o slash command /implementar-spec do projeto SDD1
d7919f9	Adiciona esqueleto do projeto Controle de Gastos e Aula 1 (transacoes)

Projeto pronto para uso

RODANDO LOCALMENTE

- `~/Projetos Claude/controle-de-gastos-pessoais-api`, repositório próprio
- `uvicorn app.main:app --reload`
- Documentação interativa em `/docs`; dashboard em `/` ou `/dashboard`
- `pytest` — 33 testes, isolados em banco temporário
- Deploy: `python -m app.main`, config via `HOST / PORT / DATABASE_PATH`

ENDPOINTS DISPONÍVEIS

POST	/transacoes
GET	/transacoes (+ filtros)
PUT	/transacoes/{id}
DELETE	/transacoes/{id}
POST	/categorias
GET	/categorias
GET	/saldo
GET	/resumo?mes=YYYY-MM
GET	/export.csv (+ filtros)
GET	/ (dashboard)
GET	/dashboard

Projeto concluído — 6/6 aulas implementadas

- 1 Deploy real numa plataforma** (Render, Railway, Fly.io etc.) — o app já lê `HOST` / `PORT` / `DATABASE_PATH` do ambiente; falta só apontar uma plataforma e configurar essas variáveis lá.
- 2 Extensões avançadas fora de escopo das 6 aulas** (spec da Aula 6 as lista explicitamente): autenticação, multiusuário, banco externo.
- 3 Persistência do SQLite em deploy** — a maioria das plataformas de PaaS tem disco efêmero; se o app for hospedado de fato, vale revisar se `DATABASE_PATH` aponta para um volume persistente.